

3. Filter 개념과 종류 익히기

Spring MVC 요청은 `DispatcherServlet`에 도달하기 전에 서블릿 필터 체인을 먼저 통과한다. 이 글에서는 `spring-web` 모듈이 기본으로 제공하는 **내장 필터 5종**이 각각 어떤 문제를 해결하는지, 그리고 사용할 때 주의할 점은 무엇인지 정리한다.

spring-web의 내장 필터는 다음 5종이다.

필터	해결하는 문제
<code>FormContentFilter</code>	POST가 아닌 메서드(PUT/PATCH/DELETE)의 폼 데이터 파싱
<code>ForwardedHeaderFilter</code>	프록시를 거치며 잃어버린 원본 요청 정보(호스트, 포트, 스킴 등) 복원
<code>ShallowEtagHeaderFilter</code>	응답 본문 기반 ETag 생성 및 <code>304 Not Modified</code> 처리
<code>CorsFilter</code>	필터 레벨의 CORS 처리 (Spring Security와 함께 쓸 때 권장)
<code>UrlHandlerFilter</code>	URL 끝 슬래시(trailing slash) 처리

3.1 Form Data : FormContentFilter

이 필터는 **Servlet API**의 한계를 메우기 위한 것이다. 브라우저는 HTML 폼 데이터를 HTTP GET 혹은 HTTP POST 로만 보낼 수 있지만, 브라우저가 아닌 클라이언트(모바일 앱, `curl`, 다른 서버 등)는 PUT, PATCH, DELETE 로도 폼 데이터를 보낼 수 있다.

하지만 서블릿 API 는 `ServletRequest.getParameter*()` 계열 메서드가 오직 **HTTP POST** 요청의 폼 필드만 읽을 수 있도록 규정한다. 그래서 `spring-web`은 **FormContentFilter**를 제공한다. 이 필터는 콘텐츠 타입이 `application/x-www-form-urlencoded`인 **PUT / PATCH /**

DELETE 요청을 가로채 body 에서 폼 데이터를 읽고, 요청을 래핑해서 `getParameter*()` 계열 메서드로도 그 값에 접근할 수 있게 만들어준다.

즉, **POST** 가 아닌 메서드에서도 폼 파라미터를 쓸 수 있게 해주는 필터다.

- 이 필터는 Spring 5.1 에서 기존 `HttpPutFormContentFilter` 를 대체하며 도입되었다. (`HttpPutFormContentFilter` 는 이름과 달리 PUT/PATCH만 처리했고, `FormContentFilter` 는 DELETE까지 지원한다.)
- **Spring Boot** 를 사용하면 `FormContentFilter` 가 자동으로 등록되며, `spring.mvc.formcontent.filter.enabled` 속성으로 켜고 끌 수 있다(기본값 `true`).

참고 : 이름이 비슷한 `HiddenHttpMethodFilter` 는 역할이 다르다. HTML 폼이 POST 로 보내면서 `_method=PUT` 같은 숨은 파라미터를 붙이면, 요청의 HTTP 메서드 자체를 바꿔주는 필터다. `FormContentFilter` 는 이미 **PUT/PATCH/DELETE** 로 들어온 요청의 **body** 를 파싱하는 필터다.

3.2 Forwarded Headers : ForwardedHeaderFilter

요청이 로드밸런서 같은 프록시들을 거치면 호스트, 포트, 스킴이 바뀔 수 있고, 그 결과 클라이언트 입장에서 올바른 링크를 생성하기가 어려워진다. 이를 위해 [RFC 7239](#) 가 표준 `Forwarded` 헤더를 정의하고 있고, 그 외에도 `X-Forwarded-Host`, `X-Forwarded-Port`, `X-Forwarded-`

`Proto`, `X-Forwarded-Ssl`, `X-Forwarded-Prefix`, `X-Forwarded-For` 같은 **비표준(사실상 표준, de facto) 헤더들**이 널리 쓰인다.

ForwardedHeaderFilter 는 이 헤더들을 기반으로 요청의 호스트, 포트, 스킴(그리고 컨텍스트 경로, 클라이언트 주소)을 **원본에 맞게 바꾸고**, 이후 처리에 더 이상 영향을 주지 않도록 그 헤더들을 제거한다. 이 필터는 요청을 래핑하는 방식으로 동작하므로, **원본이 아닌 수정된 요청을 다뤄야 하는 다른 필터보다 앞 순서에 배치해야** 한다.

여기서 **보안 고려사항**이 있다. 애플리케이션은 이 헤더들이 의도한 프록시가 붙인 것인지, 악의적인 클라이언트가 붙인 것인지 알 수 없다. 따라서 **신뢰 경계(trust boundary)에 있는 프록시가 외부에서 들어온 신뢰할 수 없는 `Forwarded` 헤더를 제거하도록** 설정해야 한다. 또한 필터를 `removeOnly=true` 로 설정하면 헤더를 사용하지 않고 **제거만** 하게 할 수 있다.

비동기 요청과 에러 디스패치를 지원하려면 이 필터를 `DispatcherType.ASYNC`, `DispatcherType.ERROR` 에도 매핑해야 한다. (Spring Framework 의 `AbstractAnnotationConfigDispatcherServletInitializer` 를 사용하면 필터가 모든 디스패치 타입에 자동으로 등록된다.)

Spring Boot에서는 `server.forward-headers-strategy` 속성으로 제어한다.

- `framework` : `ForwardedHeaderFilter` 를 등록해 Spring 이 헤더를 처리한다.
- `native` : 서블릿 컨테이너(예: Tomcat 의 `RemoteIpValve`)가 헤더를 처리한다.
- `none` : 헤더를 무시한다.

3.2.1 보충설명

프록시는 연결을 끊는다.

ex

https://shop.com 으로 보낸 클라이언트 요청이 있고, 실제 애플리케이션은 10.0.1.5:8080 에서 돌고 있다. 둘 사이에 로드밸런서(nginx, ALB 등)가 있다고 해보자.

여기서 프록시는 요청을 그대로 흘려보내는 것이 아니라, **받은 요청을 보고 새 요청을 만들어서 백엔드에 보낸다.** 즉, TCP 연결이 프록시에서 한번 끊기고 **두 개의 독립된 연결로 나뉜다.** (HTTPS 도 보통 프록시에서 종료(TLS termination)되고, 백엔드로는 HTTP 로 전달된다.)

```
Client ---->      프록시      ---->      백엔드
[브라우저]      [nginx, ALB]      [tomcat, spring, ..]
```

=> 각 연결은 완전히 별개다

클라이언트가 보낸 요청

```
getScheme() : https
getServerName() : shop.com
getServerPort() : 443
```

백엔드가 보는 요청

```
getScheme() : http
getServerName() : 10.0.1.5
getServerPort() : 8080
```

왜 문제가 되는 걸까?

애플리케이션이 **절대 URL** 을 만들어야 하는 순간이 있다. 리다이렉트 `Location` 헤더, 페이징 응답의 `next` 링크, 이메일에 넣을 링크 등이 그렇다. 이때 스프링은 **현재 요청 정보로 URL** 을 재조립한다.

```
ServletUriComponentsBuilder.fromCurrentRequest()  
// http://10.0.1.5:8080/orders/1 ---> 클라이언트가 접근 불가능  
한 주소
```

클라이언트 IP 도 마찬가지다. 백엔드 입장에서 `getRemoteAddr()` 는 **직 전 연결의 상대, 즉 프록시의 IP** 다.

- **Rate Limiting, 접근 로그, IP 기반 차단** : 전부 프록시 IP 하나로 뭉개져, 모든 사용자가 같은 IP 로 요청을 보낸 것처럼 보인다.

해결책 : 잃어버린 정보를 헤더로 실어 보낸다

프록시가 요청을 새로 만들 때, **원래 값을 헤더에 적어서 백엔드에 알려준다.**

헤더	의미	예시
<code>X-Forwarded-Proto</code>	원래 스킴	<code>https</code>
<code>X-Forwarded-Host</code>	원래 <code>Host</code> 헤더	<code>shop.com</code>
<code>X-Forwarded-Port</code>	원래 포트	<code>443</code>
<code>X-Forwarded-For</code>	원래 클라이언트 IP	<code>203.0.113.7</code>
<code>X-Forwarded-Prefix</code>	프록시가 잘라낸 경로	<code>/api</code>

참고로 표준인 `Forwarded` 헤더는 이 정보를 하나의 헤더에 담는다. `Forwarded:`
`for=203.0.113.7;proto=https;host=shop.com`

X-Forwarded-For 만 형식이 다르다. 각 프록시가 자신에게 요청을 보낸 상대(직전 홉)의 IP 를 뒤에 이어 붙인다.

ex

```
X-Forwarded-For : 203.0.113.7, 10.0.0.2, 10.0.0.9  
[최초 클라이언트] [   거쳐온 프록시들   ]
```

X-Forwarded-Prefix 만 성격이 다르다. 나머지는 연결이 끊겨서 잃어버린 정보인데, 이건 프록시가 의도적으로 경로를 바꿔서 생기는 문제다.

ex

```
Client -> GET /api/orders ----> 프록시  
프록시 --> GET /orders -----> 백엔드 -- 프록시가 /api 를 떼  
어냄  
X-Forwarded-Prefix : /api
```

프록시가 경로 앞부분을 라우팅 키로 쓰고 떼어버리는 구성이다. 백엔드는 `/orders` 로만 매핑하면 되니 편하지만, 링크를 만들 때 `/api` 를 다시 붙이지 않으면 클라이언트가 그 링크를 쓸 수 없다. 즉, 떼어낸 접두사를 알려주는 헤더이다.

ForwardedHeaderFilter 는 두 단계로 동작한다.

1. 헤더를 읽어 request 를 래

핑 : `getScheme()`, `getServerName()`, `getServerPort()`, `getRequestURL()`, `getContextPath()`, `getRemoteAddr()` 등이 헤더에 적힌 원본 값을 반환하도록 덮어쓴다.

2. 그 헤더들을 제거 : 다운스트림에서 같은 헤더가 또 적용되는 것을 막는다.

보안 경고의 의미

X-Forwarded-For 는 그냥 헤더이다. 즉, 누구나 붙일 수 있다.

```
ex
GET /admin HTTP/1.1
Host : shop.com
X-Forwarded-For : 127.0.0.1 [공격자가 직접 붙임]
```

백엔드가 이걸 무조건 믿으면 IP 기반 접근 제어가 그대로 뚫린다.

백엔드는 이 헤더를 프록시가 붙였는지 클라이언트가 붙였는지 구분할 수 없다. 그래서 방어는 신뢰 경계에 있는 프록시가 한다. 외부에서 들어온 `X-Forwarded-*` 헤더를 무조건 제거하거나, 자기가 본 값으로 덮어쓴다. 프록시가 이 설정을 하지 않으면, 백엔드에서 어떻게 설정하든 소용이 없다.

3.3 Shallow ETag : ShallowEtagHeaderFilter

이 필터는 얇은(shallow) ETag 를 만든다. 응답에 쓰인 콘텐츠를 캐싱한 뒤, 그 내용으로 MD5 해시를 계산해 ETag 를 생성한다. 다음 요청 때 클라이언트가 보낸 `If-None-Match` 헤더 값과 비교해 두 값이 같으면 `304 (Not Modified)` 를 반환한다.

이 방식은 네트워크 대역폭은 절약해주지만, 응답 자체는 매번 전부 계산해야 하므로 CPU 는 절약되지 않는다. (반대로 컨트롤러 단에서 `WebRequest.checkNotModified()` 등을 이용해 응답을 만들기 전에 판별하는 방식을 **deep ETag** 라고 한다.)

상태를 바꾸는 HTTP 메서드나 `If-Match`, `If-Unmodified-Since` 같은 다른 조건부 요청 헤더는 이 필터의 범위 밖이다. 실제로 이 필터는 `GET / HEAD` 요청이면서 `2xx` 응답이고, `Cache-Control: no-store` 가 아닌 경우에만 ETag 를 붙인다.

`setWriteWeakETag(true)` 로 설정하면 강한(strong) ETag 대신 약한(weak) ETag(`w/"..."`)를 생성하도록 할 수도 있다.

3.3.1 보충설명

3xx 상태 코드

코드	의미
301, 302, 303, 307, 308	다른 URL 로 가라는 뜻이며, <code>Location</code> 헤더에 목적지가 있다.
304	이미 가진 걸 쓰라는 뜻으로, 목적지가 없고 본문도 없다.

304 는 3xx(Redirection) 범주에 속하지만 실제로는 캐시 프로토콜이다. 본문이 없고 상태 줄과 헤더 몇 개가 전부이다. (대역폭 절약)

문제 : 바뀌지 않은 걸 계속 다시 받는다

클라이언트가 상품 목록 요청 -> 12KB 짜리 JSON 받음
-> 30초 뒤에 또 요청 -> 데이터가 바뀌지 않았는데 서버는 똑같은 12KB를 또 내보낸다.

여기서 서버는 클라이언트의 사정을 모른다. **HTTP** 는 **stateless** 라서 서버는 클라이언트가 뭘 갖고 있는지 기억하지 않는다. 그래서 클라이언트가 자기가 가진 것을 신고하게 만든다. 이를 조건부 요청(**Conditional Request**) 이라고 한다.

ETag : 응답 본문의 지문

그럼 클라이언트는 무엇을 신고해야 할까? 서버가 응답을 줄 때 그 응답의 짧은 식별자를 같이 준다. 이게 바로 **ETag(Entity Tag)** 이다.

ex) ETag : "a1b2c3d4"

내용이 바뀌면 값이 바뀌고, 안 바뀌면 그대로인 문자열이다. `ShallowEtagHeaderFilter` 는 응답 본문을 **MD5 로 해싱**해서 이 값을 만든다. 여기서 MD5 는 암호학적 용도가 아니라 **그냥 빠른 체크섬**이다. 위조 방어 목적이 아니라, **같은 내용이지만 판별**한다.

```
GET /products -- 첫 요청. 가진 캐시 없음
|
200 OK. ETag : "a1b2c3d4" -- 본문 전송
|
GET /products -- If-None-Match : "a1b2c3d4"
                        이 값과 일치하는 게 없으면 본문을 보내라.
                        일치하면 보내지 말라.
|
304 Not Modified -- 본문이 없으며 헤더만 있다.
```

`Last-Modified` / `If-Modified-Since` 쌍도 존재하지만, 이는 **시간 기반**이다. 시간 기반 검증은 두 가지 한계가 있다.

- `Last-Modified` 는 **초 단위** 정밀도라서, 1초 안에 여러 번 바뀐 변경은 구분하지 못한다.
- 수정 시각은 바뀌었지만 **내용은 그대로인 경우**에도 "변경됨"으로 판단해 본문을 다시 보낸다.

반면 ETag 는 **내용 기반**이라 이런 경우를 모두 정확히 다룰 수 있다. 그래서 ETag 가 더 정밀하다.

왜 CPU 는 절약되지 않는 걸까?

```
요청 -> 필터진입 -> 컨트롤러 실행 -> DB조회 -> JSON 직렬화 -> 응답
완성
-> 필터가 완성된 본문을 MD5로 해싱 -> If-None-Match와 비교
-> 같은 본문 버리고 304만 전송
```

해싱할 대상이 **응답 본문**이기 때문에, 응답을 다 만든 다음에야 비교를 할 수 있다. 따라서 컨트롤러도 돌고, 쿼리도 나가고, 직렬화도 다 한다. 대역 폭은 절약되지만, CPU 는 오히려 해싱 비용만큼 더 쓴다.

서버 부하는 그대로이고, 나가는 바이트만 줄어든다.

숨은 비용 : 응답 버퍼링

완성된 응답을 해싱하려면 **응답 전체가 메모리에 있어야 한다**. 그래서 이 필터는 `HttpServletResponse` 를 래핑 (`ContentCachingResponseWrapper`)해서, 컨트롤러가 쓰는 출력을 바로 소켓으로 흘려보내지 않고 **바이트 배열에 모은다**. 다 모인 뒤 해싱하고, 그때서야 전송한다.

그래서 **대용량 파일 다운로드나 스트리밍 응답에는 사용하면 안 된다**. 100MB 파일을 통째로 힙에 올리게 되며, 전송 시작이 응답 완성 시점까지 지연된다.

특정 요청만 제외하고 싶다

면 `ShallowEtagHeaderFilter.disableContentCaching(request)` 를 호출해 해당 요청의 버퍼링을 끌 수 있다.

3.4 CORS : CorsFilter

Spring MVC 는 컨트롤러 어노테이션(`@CrossOrigin`)을 통한 세밀한 CORS 설정을 지원하지만, **Spring Security** 와 함께 쓸 때는 **내장 CorsFilter** 에 의존하기를 권장한다.

이때 `CorsFilter` 는 **Spring Security** 의 필터 체인보다 앞 순서에 배치되어야 한다.

왜 앞에 있어야 할까? 브라우저가 보내는 **preflight** 요청 (**OPTIONS**) 에는 쿠키나 **Authorization** 헤더 같은 인증 정보가 실리지 않는다. CORS 처리가 Spring Security 뒤(= Spring MVC 단)에 있으면, preflight 요청이 인증 필터에서 먼저 **401 / 403** 으로 거부되어 컨트롤러의 **@CrossOrigin** 설정까지 도달하지 못한다. 참고로 Spring Security 에서 **http.cors(...)** 를 설정하면 **CorsConfigurationSource** 빈을 이용하는 **CorsFilter** 가 시큐리티 체인 안의 적절한 위치에 추가된다.

3.5 URL Handler : UrlHandlerFilter

컨트롤러 엔드포인트가 **URL** 경로의 끝 슬래시(trailing slash) 유무와 관계없이 매칭되기를 원할 때 쓰는 필터다. (Spring Framework 6.2 에서 추가되었다.)

예를 들어 **@GetMapping("/home")** 이 **GET /home** 과 **GET /home/** 을 모두 처리하게 하고 싶은 경우를 말한다.

배경 : Spring Framework 6.0 부터 끝 슬래시 매칭이 기본으로 비활성화되었다. 즉, **@GetMapping("/home")** 은 더 이상 **/home/** 에 자동으로 매칭되지 않는다. **UrlHandlerFilter** 는 이 동작을 필요한 경로에만 명시적으로 되살리는 수단이다.

UrlHandlerFilter 는 경로에서 끝 슬래시를 제거해 **일관된 경로 뷰를 보장**한다. 이는 **URL 기반 인가(authorization)** 판단과 **프레임워크의 요청 매핑 사이의 불일치**를 막는 데 중요하다. (예: 보안 규칙은 **/admin** 만 막고 있는데 **/admin/** 요청이 매핑 단에서 같은 엔드포인트로 처리되는 식의 틈을 없앤다.)

처리 방식은 두 가지다.

1. **리다이렉트** : 끝 슬래시가 없는 같은 경로로 HTTP 리다이렉트한다.
2. **요청 래핑** : 요청을 래핑해서 끝 슬래시를 제거한 경로로 처리한다.

예를 들어 블로그 애플리케이션에서는 `/blog/**` 경로는 `HttpStatus.PERMANENT_REDIRECT` (308) 로 리다이렉트하고, `/admin/**` 경로는 요청 래핑 방식으로 처리하도록 각각 다르게 구성할 수 있다.

```
UrlHandlerFilter urlHandlerFilter = UrlHandlerFilter
    .trailingSlashHandler("/blog/**").redirect(HttpStatus.P
    ERMANENT_REDIRECT)
    .trailingSlashHandler("/admin/**").wrapRequest()
    .build();
```

주의할 점

- 루트 경로 `"/` 는 끝 슬래시 처리 대상에서 **제외**된다.
- 타입 레벨 매핑에서는 `@RequestMapping("/")` 대신, **경로 속성이 없는 `@RequestMapping`** 을 쓰도록 안내한다.

정리

필터	핵심	주의점
<code>FormContentFilter</code>	PUT/PATCH/DELETE 폼 body 파싱	<code>application/x-www-form-urlencoded</code> 만 대상
<code>ForwardedHeaderFilter</code>	프록시 뒤에서 원본 요청 정보 복원	신뢰 경계의 프록시가 외부 헤더를 반드시 제거해야 함

필터	핵심	주의점
ShallowEtagHeaderFilter	본문 해시 ETag + 304	대역폭만 절약, CPU·메모리 비용 존재, 스트리밍 부적합
CorsFilter	필터 레벨 CORS 처리	Spring Security 체인보다 앞에 배치
UrlHandlerFilter	끝 슬래시 정규화	루트 "/" 제외, Spring 6.2+